Parallel Programming HW5

113062531 楊峻豪

I. Identify how UCP Objects (ucp_context, ucp_worker, ucp_ep) interact through the API, including at least the following functions:

A. ucp_init: ucp_init 是 UCP API 的初始化入口，主要負責建立 ucp_context。在這個階段 UCP 會根據使用者提供的參數與系統設定進行解析，確定可用的通訊資源，並決定所能支援的通訊功能與傳輸機制，包括不同的網路傳輸方式與通訊模式。經由 ucp_init 建立的 ucp_context，後續所有的通訊操作都會受到這個全域設定的約束。

B. ucp_worker_create: 在完成 ucp_init 並成功建立 ucp_context 之後，接下來需要透過 ucp_worker_create 來建立 ucp_worker。ucp_worker 是負責執行實際通訊處理的核心，涵蓋訊息的傳送、接收以及推進通訊進程等功能。每個 worker 都會綁定特定的執行緒模式，並在其內部管理進行通訊所需的各類資源與狀態。

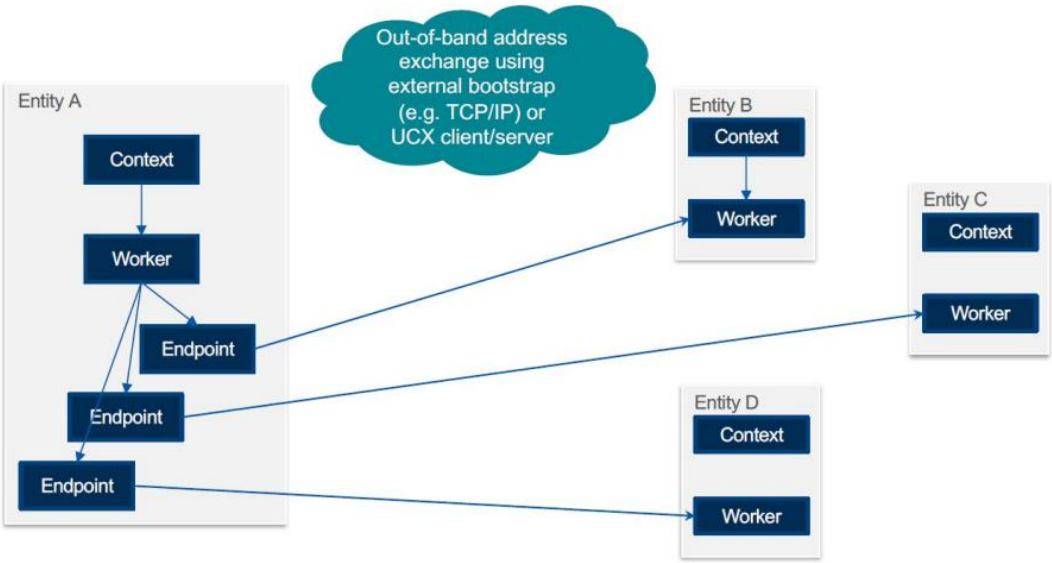
C. ucp_ep_create: 在建立 ucp_worker 之後，會透過 ucp_ep_create 與遠端的通訊對象進行連線設定。ucp_ep 用於表示本地 worker 與遠端 worker 之間的通訊關係。在建立 endpoint 時，需提供遠端的位址資訊，讓 UCP 能根據此資訊選擇最適合的傳輸方式並完成相關配置。後續的所有資料傳輸與接收操作，均透過對應的 endpoint 來執行。

II. UCX abstracts communication into three layers as below. Please provide a diagram illustrating the architectural design of UCX.

A. ucp_context: ucp_context 主要用於描述應用程式在 UCX 中的通訊環境及能力。它在執行 ucp_init 時被建立，並負責記錄可利用的通訊功能、支援的傳輸方式以及相關的全域設定。在創建 context 時，可透過參數來設定多種選項，例如支援的通訊特性（如 tag、RMA、atomic）、可使用的傳輸介面（如 InfiniBand、Ethernet、shared memory），以及資源管理或進度模式等調整選項。

B. ucp_worker: ucp_worker 是負責通訊操作的物件，它基於 ucp_context 建立。每個 worker 都會獨立管理其資源與通訊狀態，並專注於推進通訊進度（progress）、處理訊息的發送與接收。在建立 worker 時，可以根據需求配置執行緒模式（如單執行緒或多執行緒）、事件通知方式，以及其他與提升通訊效率相關的參數。

C. ucp_ep: ucp_ep 是本地 worker 與遠端 worker 之間的通訊端點。在建立 endpoint 時，必須提供遠端的位址資訊，以便 UCP 能選擇適當的傳輸方式並完成連線的設定。所有資料的傳輸與接收操作都會通過對應的 endpoint 進行處理。



III. Based on the description in HW5, where do you think the following information is loaded/created?

A. UCX_TLS

首先，在呼叫 ucp_init() 時，UCX 會讀取 UCX_TLS 環境變數，用來決定程式要啟用哪些底層通訊介面。在實作上，我從 ucp/core/ucp_context.c 開始追蹤，發現在 ucp_init_version 裡面有一個參數 ucp_config_t *config。這個 config 參數是透過 ucp_config_read 函式取得的。進一步追蹤 ucp_config_read，會發現它會呼叫 ucs_config_parser_fill_opts，而這個函式會引導我們到 ucs/config/parser.c，在裡面透過像 ucs_config_parser_set_default_values 這樣的函式，設定各種 config 的預設值。透過這個流程，UCX 將 UCX_TLS 等設定值載入到 ucp_context 中，供後續的 worker 和 endpoint 建立使用。

```
ucs_status_t ucp_init_version(unsigned api_major_version, unsigned api_minor_version,
                              const ucp_params_t *params, const ucp_config_t *config,
                              ucp_context_h *context_p)
```

```c
    if (config == NULL) {
        status = ucp_config_read(NULL, NULL, &dfl_config);
        if (status != UCS_OK) {
            goto err;
        }
        config = dfl_config;
    }
```

```c
    status = ucs_config_parser_fill_opts(config,
                                         UCS_CONFIG_GET_TABLE(ucp_config_table),
                                         config->env_prefix, 0);
```

B. TLS selected by UCX

UCX_TLS 在 ucp_context 建立時就已經被載入，但實際上選擇要使用哪個傳輸協定（TLS）往往是在建立 endpoint (ucp_ep_create) 時才決定的。當一個 worker 建立與遠端的 endpoint 時，UCX 會根據 endpoint 的位址資訊與 context 中的設定，選擇最適合的 TLS 來完成實際的連線。

I. Which files did you modify, and where did you choose to print Line 1 and Line 2?

A. Line 1

在 Print line 1 的部分，我在 ucp/core/ucp_worker.c 的 ucp_worker_get_ep_config 裡使用 ucp_config_print 來印出設定資訊。進一步追蹤，程式會進入 ucs/config/parser.c 中的 ucs_config_parser_print_opts 函式。為了印出特定的 TLS 相關資訊，我在此函式加入了第 1883 到 1886 行的程式碼。由於第 1883 行使用了 UCS_CONFIG_PRINT_TLS，我又在 ucs/config/types.h 第 94 行新增了 UCS_CONFIG_PRINT_TLS = UCS_BIT(5) 的定義。

```c
1882
1883    if (flags & UCS_CONFIG_PRINT_TLS) {
1884        ucs_config_parser_get_value(opts, fields, "TLS", str, 100 * sizeof(char));
1885        fprintf(stream, "UCX_TLS=%s\n", str);
1886    }
1887
```

```c
typedef enum {
    UCS_CONFIG_PRINT_CONFIG          = UCS_BIT(0),
    UCS_CONFIG_PRINT_HEADER          = UCS_BIT(1),
    UCS_CONFIG_PRINT_DOC             = UCS_BIT(2),
    UCS_CONFIG_PRINT_HIDDEN          = UCS_BIT(3),
    UCS_CONFIG_PRINT_COMMENT_DEFAULT = UCS_BIT(4),
    UCS_CONFIG_PRINT_TLS             = UCS_BIT(5)
} ucs_config_print_flags_t;
```

B. Line 2

在 Print line 2 的部分，我使用 ucp/core/ucp_worker.c 中的 ucp_worker_print_used_tls 函式來印出 endpoint 使用的 TLS。追蹤程式碼發現，ucp_worker_print_used_tls 內會呼叫 ucp_ep_config_name 函式，而在 src/ucp/core/ucp_ep.c 中可以看到包含我們所需的所有 TLS 資訊。因此，我直接在第 1894 行加入了印出的程式碼

```c
1844    ucp_worker_add_feature_rsc(context, key, tag_lanes_map, "tag", &strb);
1845    ucp_worker_add_feature_rsc(context, key, rma_lanes_map,
1846                               !rma_emul ? "rma" : "rma_am", &strb);
1847    ucp_worker_add_feature_rsc(context, key, amo_lanes_map,
1848                               !amo_emul ? "amo" : "amo_am", &strb);
1849    ucp_worker_add_feature_rsc(context, key, am_lanes_map, "am", &strb);
1850    ucp_worker_add_feature_rsc(context, key, stream_lanes_map, "stream", &strb);
1851    ucp_worker_add_feature_rsc(context, key, ka_lanes_map, "ka", &strb);
1852
1853    ucs_string_buffer_rtrim(&strb, "; ");
1854    printf("%s\n", ucs_string_buffer_cstr(&strb));
1855    ucs_info("%s", ucs_string_buffer_cstr(&strb));
1856 }
1857
```

```c
1917 void ucp_ep_config_name(ucp_worker_h worker, ucp_worker_cfg_index_t cfg_index,
1918                        ucs_string_buffer_t *strb)
1919 {
1920     ucp_context_h context          = worker->context;
1921     const ucp_ep_config_key_t *key = &ucs_array_elem(&worker->ep_config,
1922                                                      cfg_index).key;
1923
1924     if (!ucs_string_is_empty(context->name)) {
1925         ucs_string_buffer_appendf(strb, "%s ", context->name);
1926     }
1927
1928     if (key->flags & UCP_EP_CONFIG_KEY_FLAG_SELF) {
1929         ucs_string_buffer_appendf(strb, "self ");
1930     } else if (key->flags & UCP_EP_CONFIG_KEY_FLAG_INTRA_NODE) {
1931         ucs_string_buffer_appendf(strb, "intra-node ");
1932     } else {
1933         ucs_string_buffer_appendf(strb, "inter-node ");
1934     }
1935
1936     ucs_string_buffer_appendf(strb, "cfg#%d", cfg_index);
1937 }
```

II. How do the functions in these files call each other? Why is it designed this way?

Line 1 是透過 ucp_worker_get_ep_config 呼叫 ucp_config_print，而 ucp_config_print 又會進入 ucs_config_parser_print_opts 來列印設定。這條路徑主要用來印出整個 context 或 worker 所使用的 TLS 配置與其他全域設定。 Line 2 則是透過 ucp_worker_print_used_tls 呼叫 ucp_ep_config_name，直接取得每個 endpoint 實際使用的 TLS，並印出詳細資訊。

III. Observe when Line 1 and 2 are printed during the call of which UCP API? Line 1 和 Line 2 的資訊是在 endpoint 還沒正式建立連線前印出的，也就是在 ucp_ep_init_create_wireup 呼叫 ucp_worker_get_ep_config 的過程中取得。這樣做可以在實際通訊之前，先確認每個 endpoint 將使用的 TLS 與配置，方便除錯與資源管理。

IV. Does it match your expectations for questions 1-3? Why?

我們追蹤了 Line 1 與 Line 2 的函式呼叫路徑，發現 UCX 採用層級化設計，從 ucp_context → ucp_worker → ucp_ep，分別列印全域配置與 endpoint 使用的 TLS，符合我對 UCX 架構與資訊分層的理解。 Line 1 和 Line 2 在建立 endpoint（wireup）之前被印出，也符合我對「先取得資源與配置，再建立實際通訊通道」的預期。

V. In implementing the features, we see variables like lanes, tl_rsc, tl_name, tl_device, bitmap, iface, etc., used to store different Layer's protocol information. Please explain what information each of them stores.

A. lanes: 示 endpoint 中可用的傳輸通道，每個 lane 對應一個底層傳輸資

源（例如不同的 network transport 或 shared-memory path）

B. tl_rsc: 底層傳輸資源，包含 transport 類型與硬體資源。

C. tl_name: 傳輸層名稱，如 tcp、ud。

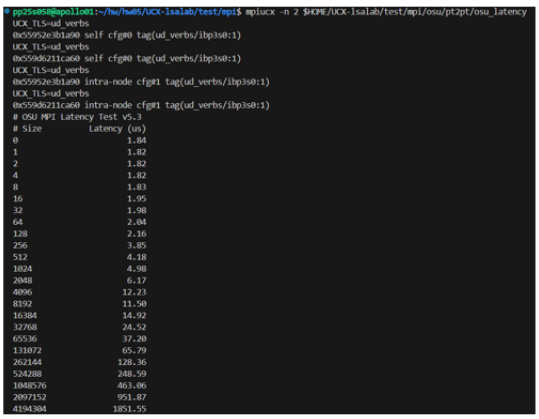
D. tl_device: 使用的具體硬體裝置，例如 eth0、ibp3s0。

E. bitmap: 位元圖，表示可用的傳輸資源。

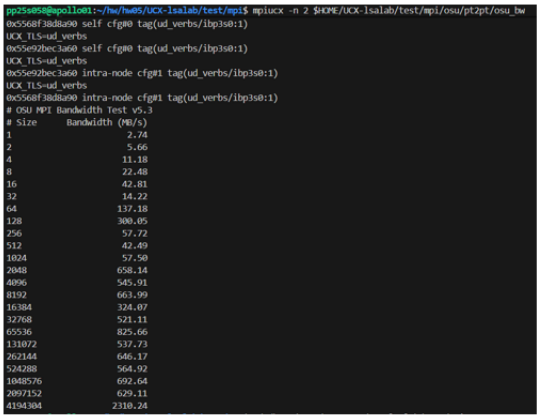
F. iface: 底層介面對象，用於實際資料傳送與接收。

I. Optimization

目前的 OpenMPI 被強制要求透過 InfiniBand 網路卡 (verbs) 來進行通訊。即使兩個 Process 都在同一台機器上，資料也會繞到網路卡再回來，因為經過了硬體層導致延遲（Latency）過高。頻寬也受限於網卡的物理速度。透過 export UCX_TLS=shm,self,ud_verbs 可以指定使用 share memory 直接操作記憶體位址，完全不需要經過網路硬體。



```
pp25s050@apollo01:~/hw/hw05/UCX-lsalab/test/mpi$ mpiucx -n 2 $HOME/UCX-lsalab/test/mpi/osu/pt2pt/osu_latency
UCX_TLS=ud_verbs
0x55952e3b1a90 self cfg#0 tag(ud_verbs/ibp3s0:1)
UCX_TLS=ud_verbs
0x559d6211ca60 self cfg#0 tag(ud_verbs/ibp3s0:1)
UCX_TLS=ud_verbs
0x55952e3b1a90 intra-node cfg#1 tag(ud_verbs/ibp3s0:1)
UCX_TLS=ud_verbs
0x559d6211ca60 intra-node cfg#1 tag(ud_verbs/ibp3s0:1)
# OSU MPI Latency Test v5.3
# Size          Latency (us)
0                       1.84
1                       1.82
2                       1.82
4                       1.82
8                       1.83
16                      1.95
32                      1.98
64                      2.04
128                     2.16
256                     3.85
512                     4.18
1024                    4.98
2048                    6.17
4096                   12.23
8192                   11.50
16384                  14.92
32768                  24.52
65536                  37.20
131072                 65.79
262144                128.36
524288                248.59
1048576               463.06
2097152               951.87
4194304              1851.55
```



```
pp25s050@apollo01:~/hw/hw05/UCX-lsalab/test/mpi$ mpiucx -n 2 $HOME/UCX-lsalab/test/mpi/osu/pt2pt/osu_bw
0x5568f38d8a90 self cfg#0 tag(ud_verbs/ibp3s0:1)
UCX_TLS=ud_verbs
0x55e92bec3a60 self cfg#0 tag(ud_verbs/ibp3s0:1)
UCX_TLS=ud_verbs
0x55e92bec3a60 intra-node cfg#1 tag(ud_verbs/ibp3s0:1)
UCX_TLS=ud_verbs
0x5568f38d8a90 intra-node cfg#1 tag(ud_verbs/ibp3s0:1)
# OSU MPI Bandwidth Test v5.3
# Size      Bandwidth (MB/s)
1                       2.74
2                       5.66
4                      11.18
8                      22.48
16                     42.81
32                     14.22
64                    137.18
128                   300.05
256                    57.72
512                    42.49
1024                   57.50
2048                  658.14
4096                  545.91
8192                  663.99
16384                 324.07
32768                 521.11
65536                 825.66
131072                537.73
262144                646.17
524288                564.92
1048576               692.64
2097152               629.11
4194304              2310.24
```

從新的實驗數據可以看到 latency 變低了、而 bandwith 也變高了。

```
pp25s058@apollo01:~/UCX-1salab/test/mpi$ mpiucx -n 2 $HOME/UCX-1salab/test/mpi/osu/pt2pt/osu_latency
UCX_TLS=shm,self,ud_verbs
0x55ce8e26ea60 self cfg#0 tag(self/memory cma/memory)
UCX_TLS=shm,self,ud_verbs
0x561045e00a90 self cfg#0 tag(self/memory cma/memory)
UCX_TLS=shm,self,ud_verbs
0x55ce8e26ea60 intra-node cfg#1 tag(sysv/memory cma/memory)
UCX_TLS=shm,self,ud_verbs
0x561045e00a90 intra-node cfg#1 tag(sysv/memory cma/memory)
# OSU MPI Latency Test v5.3
# Size          Latency (us)
0                       0.21
1                       0.21
2                       0.21
4                       0.21
8                       0.21
16                      0.21
32                      0.24
64                      0.25
128                     0.36
256                     0.39
512                     0.43
1024                    0.51
2048                    0.67
4096                    1.01
8192                    1.94
16384                   3.06
32768                   5.05
65536                   8.99
131072                 17.32
262144                 37.24
524288                 69.51
1048576               126.93
2097152               279.13
4194304               971.65
```

```
pp25s058@apollo01:~/UCX-1salab/test/mpi$ mpiucx -n 2 $HOME/UCX-1salab/test/mpi/osu/pt2pt/osu_bw
UCX_TLS=shm,self,ud_verbs
0x55dbcc5c4a90 self cfg#0 tag(self/memory cma/memory)
UCX_TLS=shm,self,ud_verbs
0x55766a40ea60 self cfg#0 tag(self/memory cma/memory)
UCX_TLS=shm,self,ud_verbs
0x55766a40ea60 intra-node cfg#1 tag(sysv/memory cma/memory)
UCX_TLS=shm,self,ud_verbs
0x55dbcc5c4a90 intra-node cfg#1 tag(sysv/memory cma/memory)
# OSU MPI Bandwidth Test v5.3
# Size      Bandwidth (MB/s)
1                       9.34
2                      18.79
4                      37.71
8                      75.54
16                    137.57
32                    296.49
64                    597.05
128                   656.61
256                  1242.54
512                  2333.98
1024                 3740.65
2048                 5684.07
4096                 7809.76
8192                 9890.91
16384                4851.37
32768                6640.47
65536                8467.02
131072               8942.27
262144               8372.50
524288               8373.01
1048576              8151.41
2097152              8280.06
4194304              7416.71
```

II. Conclusion

A. What have you learned from this homework?

從這次作業中，我學到了 UCX 的層級化設計與通訊流程，包括 ucp_context、ucp_worker 與 ucp_ep 三個核心物件如何協作，以及它們在建立通訊前如何取得資源與配置。我也透過追蹤原始碼理解了 UCX_TLS 等環境變數如何在 context 載入，並在 worker 與 endpoint 中被實際使用。

B. How long did you spend on the assignment?

大約五天